

Section A: Conceptual Questions

- 1. You are testing a web application where user details are stored in a relational database. Explain how understanding database types helps you decide what kind of testing approach is needed.**

Understanding database types helps tester know **how data is stored and managed**. This helps us choose the right testing approach.

For a relational database, data is stored in table(rows and column). During testing, we check:

- Data is saved correctly in the database.
- Data is update and deleted correctly.
- Tables are connected properly(relationships).
- No duplicate or missing data.
- Data shown in the application matches the database.

- 2. While validating a search feature, why would filtering data using conditions be more effective than validating random records?**

Filtering data using condition is more effective because it check specific and expected results. It helps us quickly find the correct records and verify that the search feature is working properly. Random records may miss important cases or bugs.

EXAMPLE:

If you search for “**john**”, filtering checks only the records with the name **john**. Randomly checking records may not confirm whether the search is working correctly.

- 3. During test execution, incorrect user data appears on the user interface. Explain how database validation can help identify the root cause.**

Database validation helps identify the root cause by comparing the data stored in the database with the data shown on the user interface.

- If the database contains the correct data but the UI shows incorrect data, the issue is in the application or UI.
- If the database also contains incorrect data, the issue is in the database, backend, or data-saving process.

This helps testers find where the problem actually exists and fix it faster.

Example:

A user's email in the database is `abc@gmail.com`, but the UI shows `xyz@gmail.com`. By checking the database, we can determine whether the issue is in the database or in the application's display logic.

4. A tester directly updates user status in the database during testing. Explain when this approach is acceptable and when it is risky.

When it's okay

- If you are working in a **test database** (not real users).
- If you just need to **set up test data quickly** (like making a user “active” or “blocked” to check how the system reacts).
- If it's only for **debugging or temporary testing**.

When it's risky

- If you do it in the **real (production) database** where actual users exist.
- If you **skip the normal application steps** (like login, upgrade, or deactivate) because you might miss bugs in those flows.
- If it breaks **rules, security, or audit logs** since direct changes don't leave a proper record.
- If it causes **wrong or inconsistent data** that the system can't handle.

5. Explain how database constraints reduce the number of defects reaching production.

Database constraints help keep the data correct and valid. They prevent invalid or duplicate data from being stored in the database. This reduces errors and prevents many defects from reaching production.

Example:

If the **email** field has a **UNIQUE** constraint, two users cannot register with the same email address. This prevents duplicate data and reduces defects.

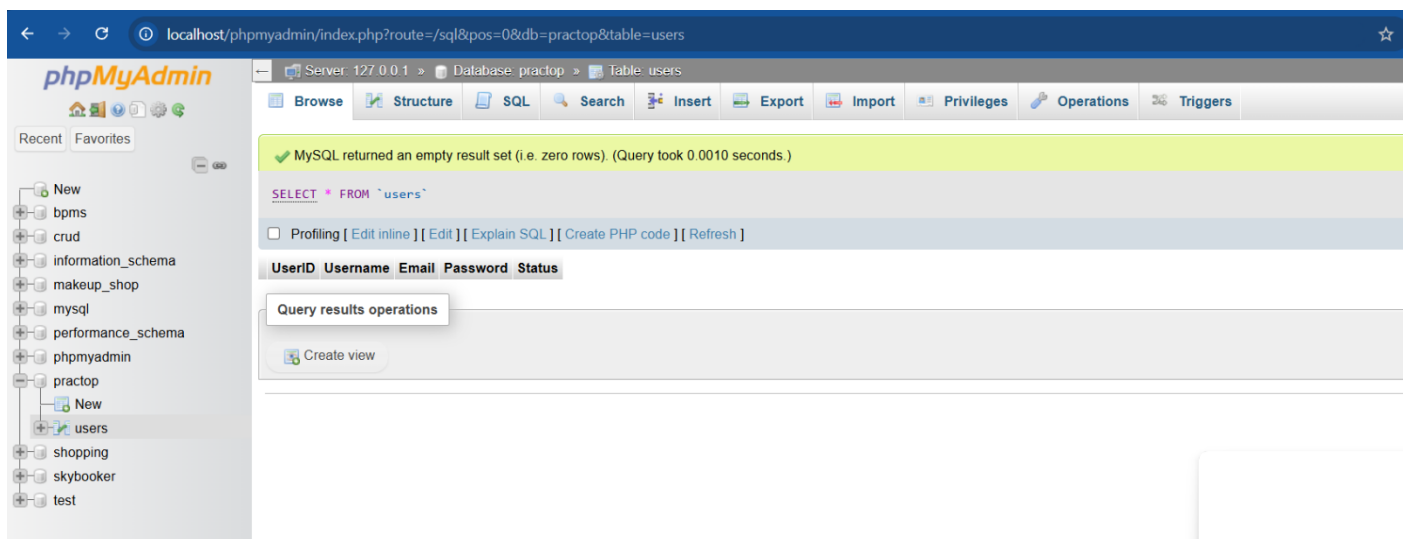
Examples of constraints:

- **Primary Key** – Prevents duplicate IDs.
- **NOT NULL** – Prevents empty required fields.
- **UNIQUE** – Prevents duplicate values like email addresses.
- **Foreign Key** – Maintains correct relationships between tables.

Section B: Practical SQL Tasks

1. **Table Creation: Create a Users table with appropriate data types (e.g., INT, VARCHAR, ENUM). Fields must include: User Identifier (Primary Key), Username, Email, Password, and Status**

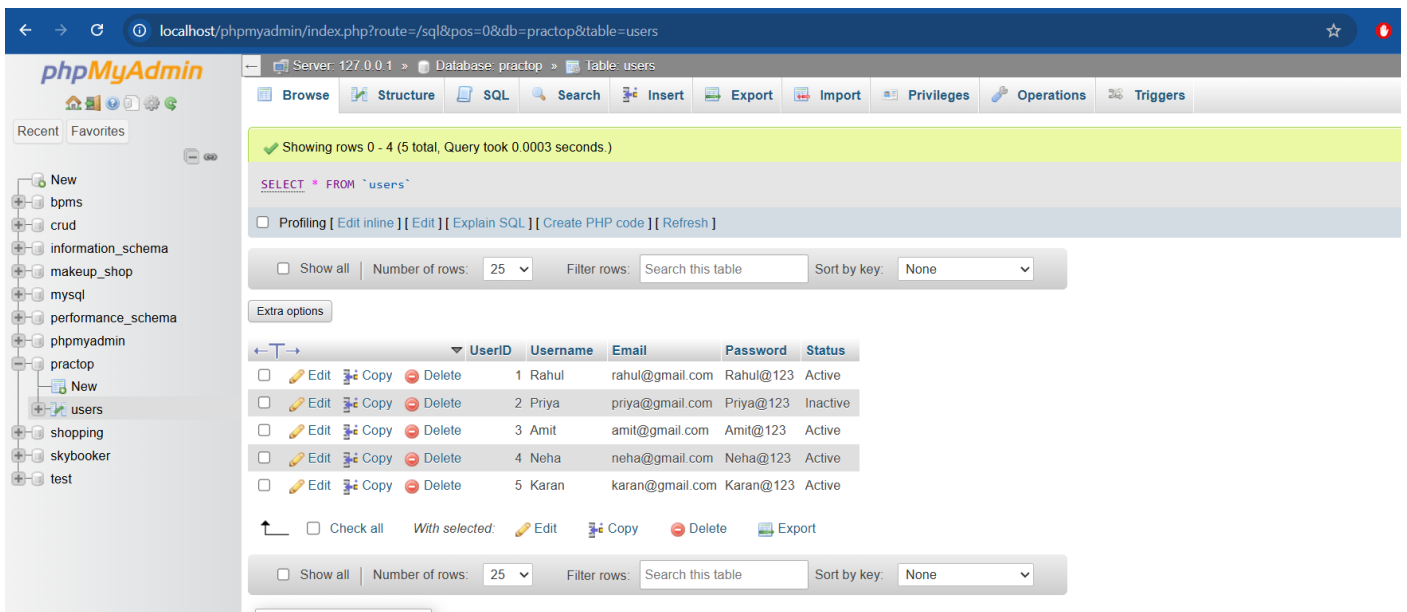
```
CREATE TABLE Users (
  UserID INT PRIMARY KEY AUTO_INCREMENT,
  Username VARCHAR(100) NOT NULL,
  Email VARCHAR(100) NOT NULL UNIQUE,
  Password VARCHAR(255) NOT NULL,
  Status ENUM('Active', 'Inactive') NOT NULL
);
```



2. **Data insertion:** write **INSERT INTO** statements to **add 5 test user**. Ensure a mix of data for testing, including at least one “inactive” status and one user with a last login date older than a year.

```
INSERT INTO Users (Username, Email, Password, Status)
VALUES
```

```
('Rahul', 'rahul@gmail.com', 'Rahul@123', 'Active'),
('Priya', 'priya@gmail.com', 'Priya@123', 'Inactive'),
('Amit', 'amit@gmail.com', 'Amit@123', 'Active'),
('Neha', 'neha@gmail.com', 'Neha@123', 'Active'),
('Karan', 'karan@gmail.com', 'Karan@123', 'Active');
```



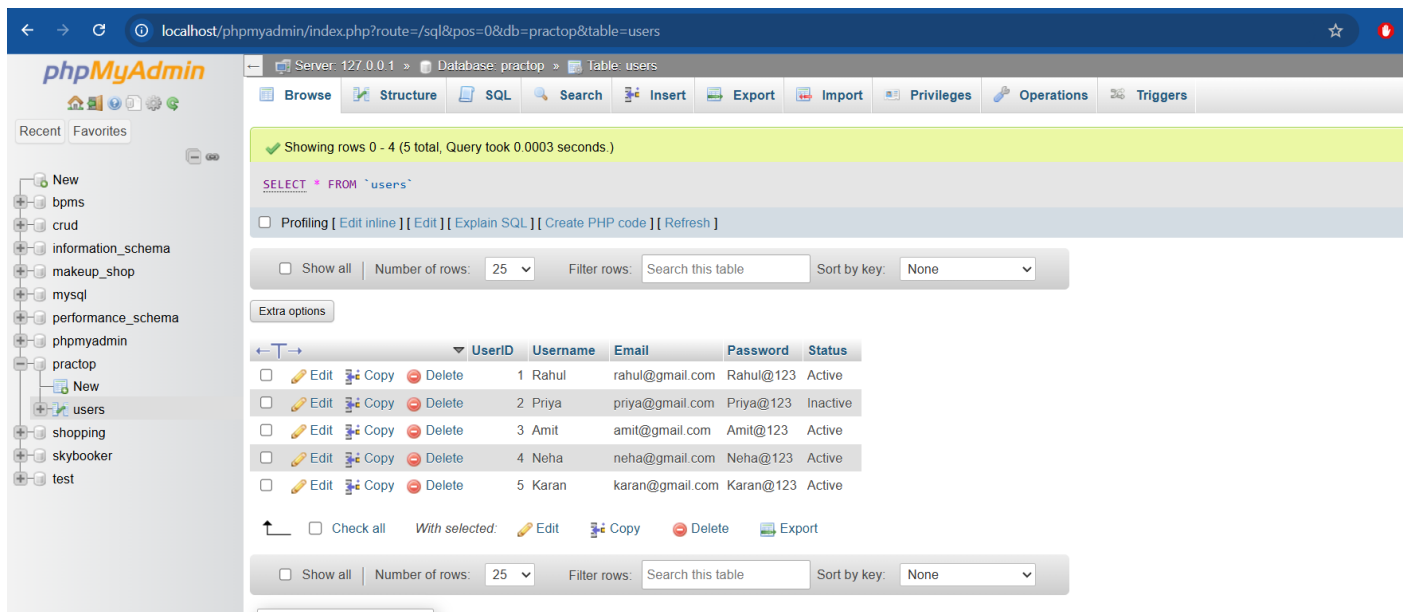
The screenshot shows the phpMyAdmin interface for a database named 'practop'. The 'users' table is selected, and the SQL query 'SELECT * FROM `users`' is executed. The table contains 5 rows of data, including one inactive user (Priya) and four active users (Rahul, Amit, Neha, Karan).

UserID	Username	Email	Password	Status
1	Rahul	rahul@gmail.com	Rahul@123	Active
2	Priya	priya@gmail.com	Priya@123	Inactive
3	Amit	amit@gmail.com	Amit@123	Active
4	Neha	neha@gmail.com	Neha@123	Active
5	Karan	karan@gmail.com	Karan@123	Active

3. **Data Retrieval (10 Queries):** Write 10 **SELECT** queries demonstrating:

- **Filtering:** Using **WHERE** for specific statuses.
- **Pattern Matching:** Using **LIKE** (e.g., finding emails ending in @gmail.com).
- **Sorting:** Using **ORDER BY** for usernames or identifiers.
- **Conditions:** Using **AND/OR** and **IS NOT NULL**.

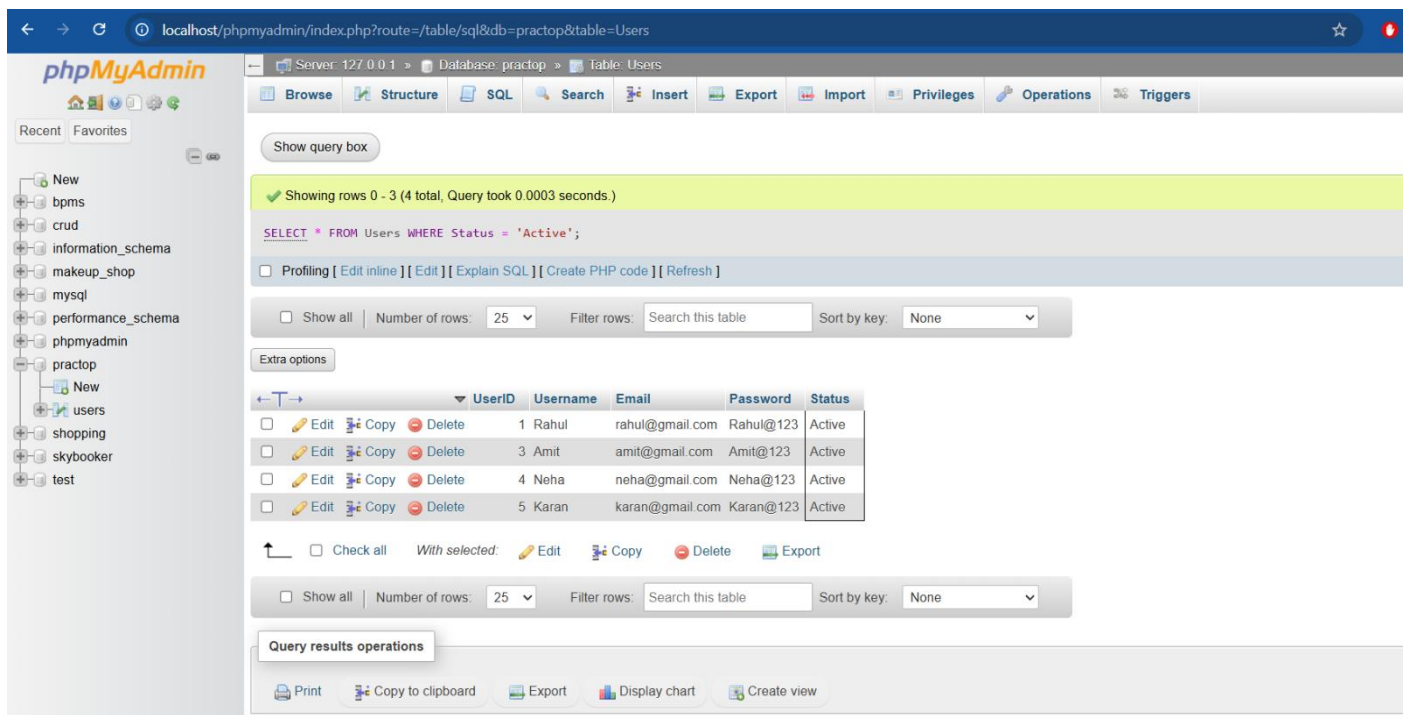
1. Display all users: `SELECT * FROM Users;`



The screenshot shows the phpMyAdmin interface for the 'practop' database, specifically the 'users' table. The query 'SELECT * FROM `users`' has been executed, displaying 5 rows. The table structure is as follows:

UserID	Username	Email	Password	Status
1	Rahul	rahul@gmail.com	Rahul@123	Active
2	Priya	priya@gmail.com	Priya@123	Inactive
3	Amit	amit@gmail.com	Amit@123	Active
4	Neha	neha@gmail.com	Neha@123	Active
5	Karan	karan@gmail.com	Karan@123	Active

2. Show only Active users (WHERE): `SELECT * FROM Users WHERE Status = 'Active';`

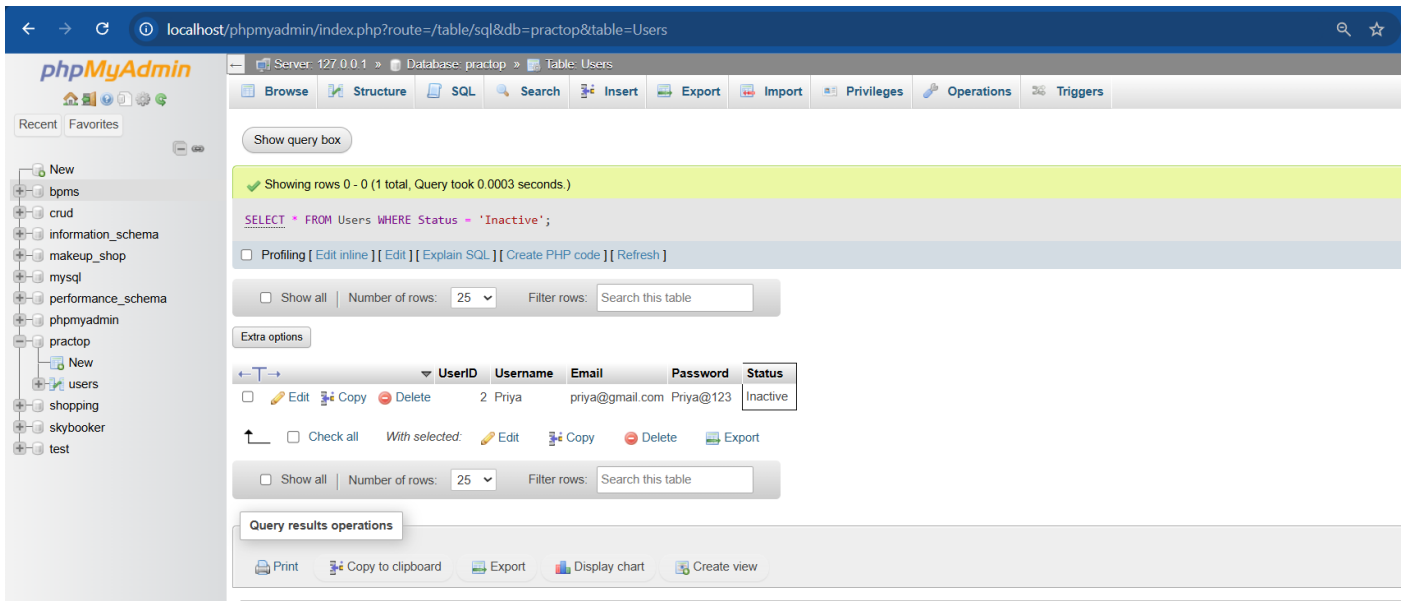


The screenshot shows the phpMyAdmin interface for the 'practop' database, specifically the 'Users' table. The query 'SELECT * FROM Users WHERE Status = 'Active';' has been executed, displaying 4 rows of active users. The table structure is as follows:

UserID	Username	Email	Password	Status
1	Rahul	rahul@gmail.com	Rahul@123	Active
3	Amit	amit@gmail.com	Amit@123	Active
4	Neha	neha@gmail.com	Neha@123	Active
5	Karan	karan@gmail.com	Karan@123	Active

Below the table, the 'Query results operations' section is visible, containing links for Print, Copy to clipboard, Export, Display chart, and Create view.

3. Show only Inactive users (WHERE) : **SELECT * FROM Users WHERE Status = 'Inactive';**



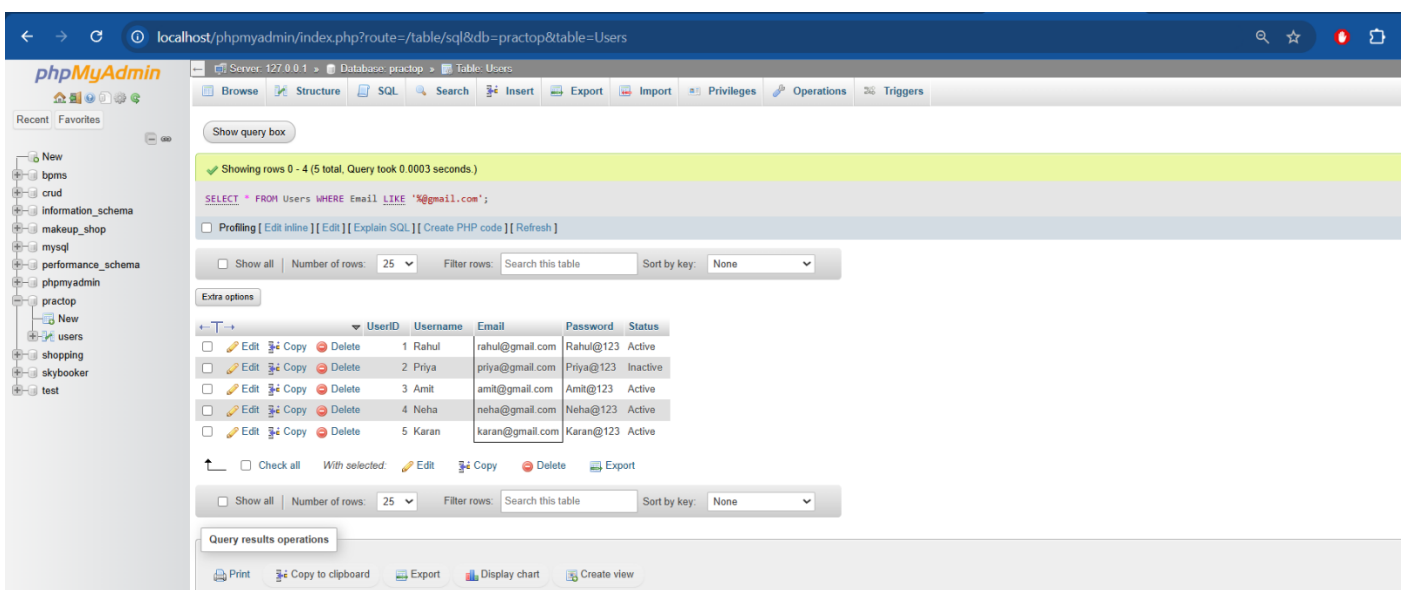
The screenshot shows the phpMyAdmin interface with the 'Users' table selected. The SQL query `SELECT * FROM Users WHERE Status = 'Inactive';` has been executed, resulting in one row being displayed. The row details are as follows:

UserID	Username	Email	Password	Status
2	Priya	priya@gmail.com	Priya@123	Inactive

4. Find users whose email ends with @gmail.com (LIKE):

SELECT * FROM Users

WHERE Email LIKE '%@gmail.com';



The screenshot shows the phpMyAdmin interface with the 'Users' table selected. The SQL query `SELECT * FROM Users WHERE Email LIKE '%@gmail.com';` has been executed, resulting in five rows being displayed. The row details are as follows:

UserID	Username	Email	Password	Status
1	Rahul	rahul@gmail.com	Rahul@123	Active
2	Priya	priya@gmail.com	Priya@123	Inactive
3	Amit	amit@gmail.com	Amit@123	Active
4	Neha	neha@gmail.com	Neha@123	Active
5	Karan	karan@gmail.com	Karan@123	Active

5. Find usernames starting with 'R' (LIKE):

```
SELECT * FROM Users
```

```
WHERE Username LIKE 'R%';
```

The screenshot shows the phpMyAdmin interface for the 'practop' database. The 'Users' table is selected. The SQL query entered is `SELECT * FROM Users WHERE Username LIKE 'R%';`. The results show one row: User ID 1, Username 'Rahul', Email 'rahul@gmail.com', Password 'Rahul@123', and Status 'Active'.

UserID	Username	Email	Password	Status
1	Rahul	rahul@gmail.com	Rahul@123	Active

6. Sort users by Username in ascending order (ORDER BY):

```
SELECT * FROM Users
```

```
ORDER BY Username ASC;
```

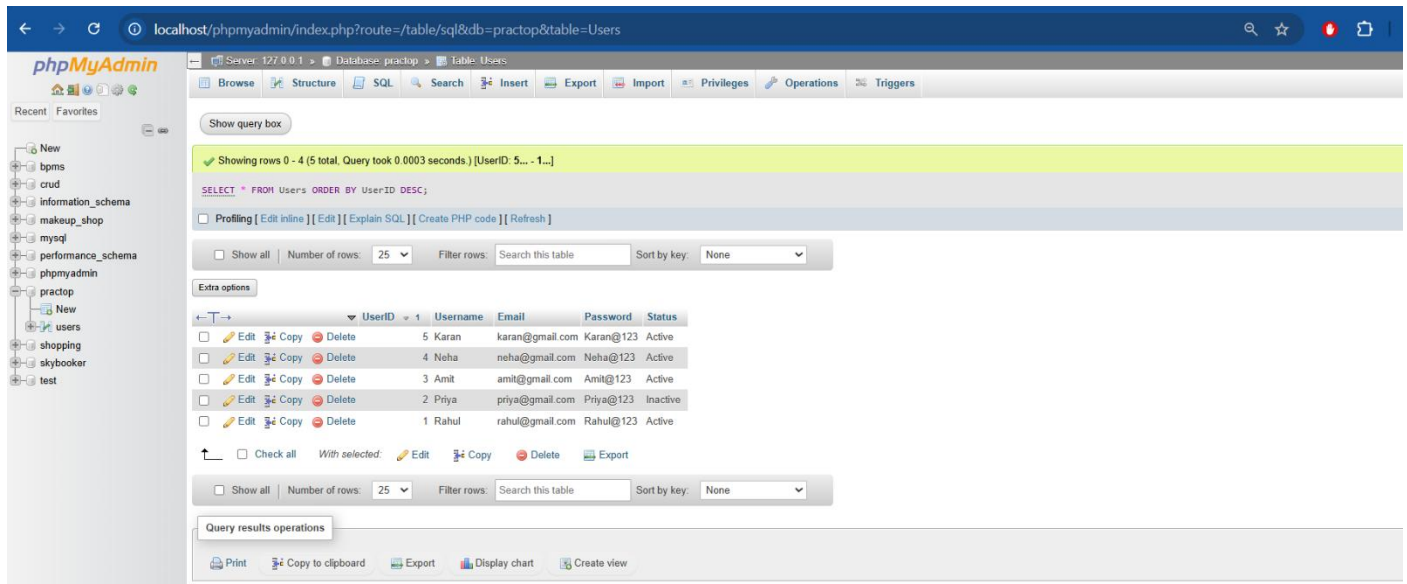
The screenshot shows the phpMyAdmin interface for the 'practop' database. The 'Users' table is selected. The SQL query entered is `SELECT * FROM Users ORDER BY Username ASC;`. The results show five rows, sorted by username in ascending order: Amit, Karan, Neha, Priya, and Rahul.

UserID	Username	Email	Password	Status
3	Amit	amit@gmail.com	Amit@123	Active
5	Karan	karan@gmail.com	Karan@123	Active
4	Neha	neha@gmail.com	Neha@123	Active
2	Priya	priya@gmail.com	Priya@123	Inactive
1	Rahul	rahul@gmail.com	Rahul@123	Active

7. Sort users by UserID in descending order (ORDER BY):

SELECT * FROM Users

ORDER BY UserID DESC;



The screenshot shows the phpMyAdmin interface for the 'practop' database, specifically the 'Users' table. The SQL query entered is `SELECT * FROM Users ORDER BY UserID DESC;`. The results show 5 rows, sorted by UserID in descending order. The table has columns: UserID, Username, Email, Password, and Status.

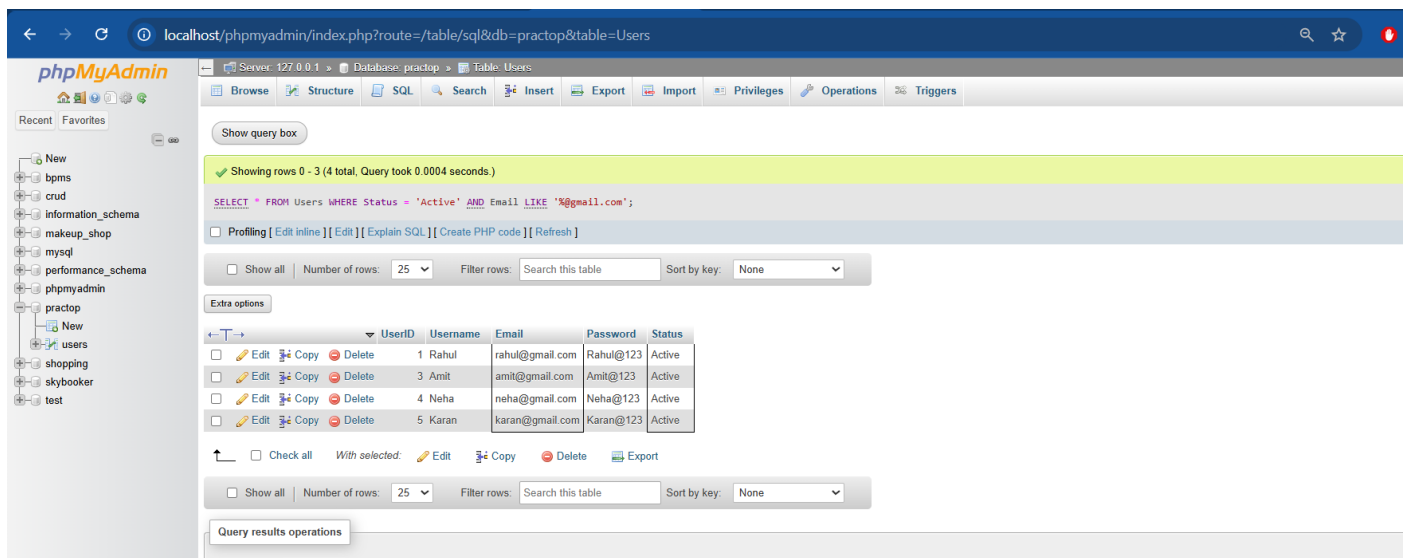
	UserID	Username	Email	Password	Status
☐	5	Karan	karan@gmail.com	Karan@123	Active
☐	4	Neha	neha@gmail.com	Neha@123	Active
☐	3	Amit	amit@gmail.com	Amit@123	Active
☐	2	Priya	priya@gmail.com	Priya@123	Inactive
☐	1	Rahul	rahul@gmail.com	Rahul@123	Active

8. Use AND condition:

SELECT * FROM Users

WHERE Status = 'Active'

AND Email LIKE '%@gmail.com';



The screenshot shows the phpMyAdmin interface for the 'practop' database, specifically the 'Users' table. The SQL query entered is `SELECT * FROM Users WHERE Status = 'Active' AND Email LIKE '%@gmail.com';`. The results show 4 rows, filtered by Status = 'Active' and Email LIKE '%@gmail.com'. The table has columns: UserID, Username, Email, Password, and Status.

	UserID	Username	Email	Password	Status
☐	1	Rahul	rahul@gmail.com	Rahul@123	Active
☐	3	Amit	amit@gmail.com	Amit@123	Active
☐	4	Neha	neha@gmail.com	Neha@123	Active
☐	5	Karan	karan@gmail.com	Karan@123	Active

9. Use OR condition:

SELECT * FROM Users

WHERE Username = 'Rahul'

OR Username = 'Priya';

The screenshot shows the phpMyAdmin interface with the 'Users' table selected. The query executed is `SELECT * FROM Users WHERE Username = 'Rahul' OR Username = 'Priya';`. The results show two rows: one for 'Rahul' (Active) and one for 'Priya' (Inactive).

UserID	Username	Email	Password	Status
1	Rahul	rahul@gmail.com	Rahul@123	Active
2	Priya	priya@gmail.com	Priya@123	Inactive

10. Use IS NOT NULL:

SELECT * FROM Users

WHERE Email IS NOT NULL;

The screenshot shows the phpMyAdmin interface with the 'Users' table selected. The query executed is `SELECT * FROM Users WHERE Email IS NOT NULL;`. The results show five rows, all with non-null email addresses.

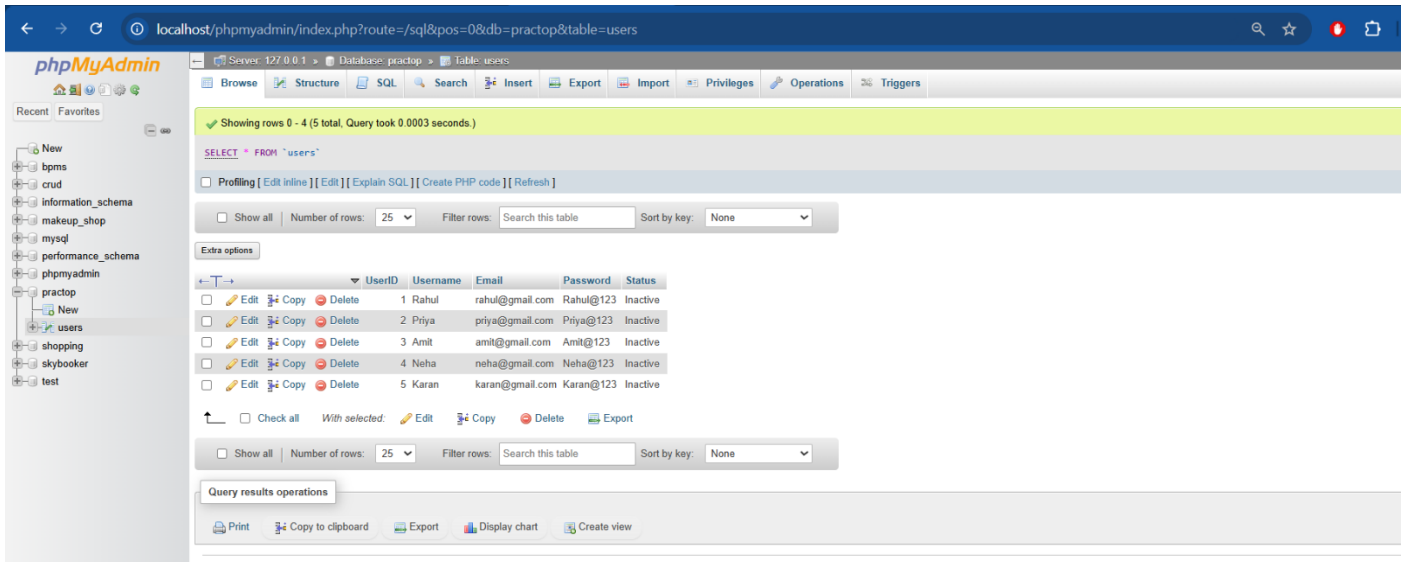
UserID	Username	Email	Password	Status
1	Rahul	rahul@gmail.com	Rahul@123	Active
2	Priya	priya@gmail.com	Priya@123	Inactive
3	Amit	amit@gmail.com	Amit@123	Active
4	Neha	neha@gmail.com	Neha@123	Active
5	Karan	karan@gmail.com	Karan@123	Active

4. **Data updates:** Write an **UPDATE** statement to change the status of specific Users(e.g, setting “pending” account to “ inactive” based on a condition).

UPDATE Users

SET Status = 'Inactive'

WHERE Status = 'Active';



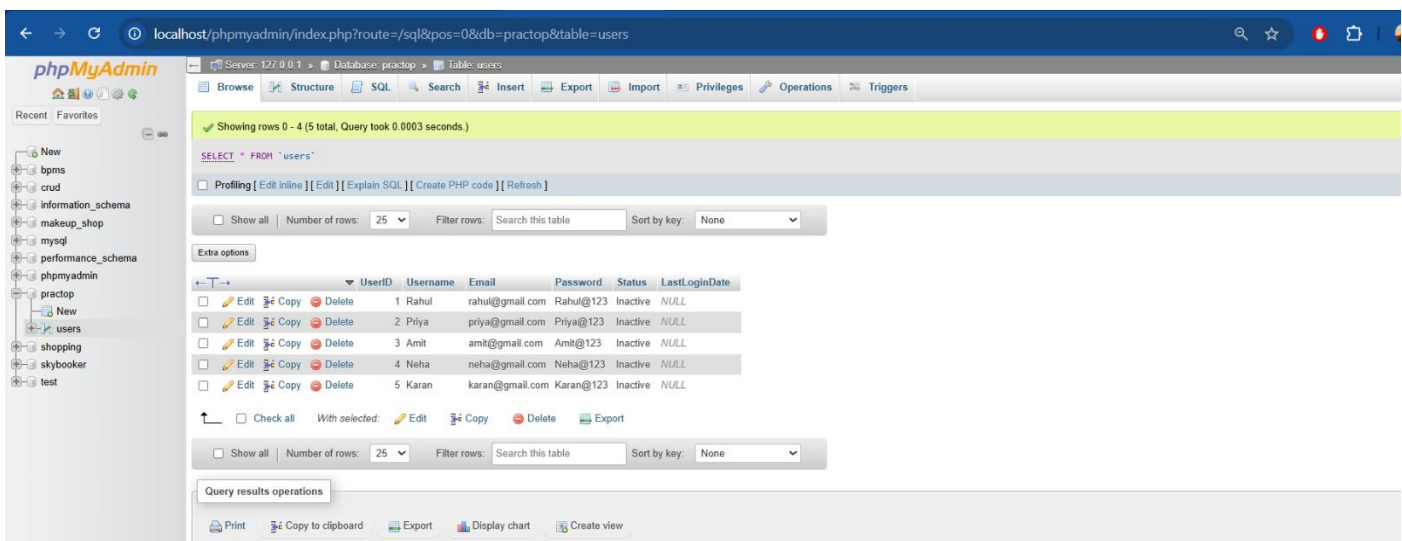
The screenshot shows the phpMyAdmin interface for the 'practop' database, specifically the 'users' table. The table contains 5 rows of data, all with a status of 'Inactive'. The columns are UserID, Username, Email, Password, and Status.

UserID	Username	Email	Password	Status
1	Rahul	rahul@gmail.com	Rahul@123	Inactive
2	Priya	priya@gmail.com	Priya@123	Inactive
3	Amit	amit@gmail.com	Amit@123	Inactive
4	Neha	neha@gmail.com	Neha@123	Inactive
5	Karan	karan@gmail.com	Karan@123	Inactive

5. **Data Deletion:** Write a **DELETE** query to remove users who haven't logged in for over a year.
Tip: Use date functions like **DATEDIFF()** or **CURRENT_DATE - INTERVAL '1 year'** depending on your SQL dialect.

ALTER TABLE Users

ADD LastLoginDate DATE;



The screenshot shows the phpMyAdmin interface for the 'practop' database, specifically the 'users' table. The table now includes a 'LastLoginDate' column, which is NULL for all users. The columns are UserID, Username, Email, Password, Status, and LastLoginDate.

UserID	Username	Email	Password	Status	LastLoginDate
1	Rahul	rahul@gmail.com	Rahul@123	Inactive	NULL
2	Priya	priya@gmail.com	Priya@123	Inactive	NULL
3	Amit	amit@gmail.com	Amit@123	Inactive	NULL
4	Neha	neha@gmail.com	Neha@123	Inactive	NULL
5	Karan	karan@gmail.com	Karan@123	Inactive	NULL

Section C: Mini Project – End-to-End Database Validation Context

- 1. Test Planning Document: Prepare a Test Plan focusing on database testing scope, risks, test data strategy, and validation approach.**

Objective

To verify that user data is correctly stored, updated, retrieved, and deleted in the database.

Scope

- Verify user data is stored correctly.
- Verify data can be retrieved correctly.
- Verify data is updated correctly.
- Verify data is deleted correctly.
- Verify database constraints such as Primary Key, Unique, and NOT NULL.

Risks

- Incorrect or missing data in the database.
- Duplicate email addresses.
- Data mismatch between the application and the database.
- Incorrect update or deletion of records.

Test Data Strategy

- Use valid user data.
- Use invalid user data.
- Use duplicate email addresses.
- Test Active and Inactive users.
- Test NULL and empty values where applicable.

Validation Approach

- Compare data displayed in the application with the database.
- Execute SQL queries (SELECT, INSERT, UPDATE, DELETE).
- Verify data integrity and database constraints.

- Ensure CRUD (Create, Read, Update, Delete) operations work correctly.
- Check that no duplicate or invalid data is stored.

2. Requirements Traceability Matrix: Create a Requirements Traceability Matrix linking user requirements to database validation queries.

Req ID	User Requirement	Database Validation Query	Expected Result	Status
1	User registration data should be saved successfully.	SELECT * FROM Users WHERE Email='rahul@gmail.com';	User record is available in the database.	Pass
2	Email address should be unique.	SELECT Email, COUNT(*) FROM Users GROUP BY Email HAVING COUNT(*) > 1;	No duplicate email records found.	Pass
3	User status should be stored correctly.	SELECT * FROM Users WHERE Status='Active';	Active users are displayed correctly.	Pass
4	Inactive users should be identified correctly.	SELECT * FROM Users WHERE Status='Inactive';	Only inactive users are displayed.	Pass
5	User details should be updated successfully.	UPDATE Users SET Status='Inactive' WHERE UserID=1;SELECT * FROM Users WHERE UserID=1;	Status is updated successfully.	Pass
6	Users who have not logged in for over a year should be removed.	DELETE FROM Users WHERE LastLoginDate < DATE_SUB(CURDATE(), INTERVAL 1 YEAR);	Old user records are deleted.	Pass
7	Search users by email domain.	SELECT * FROM Users WHERE Email LIKE '%@gmail.com';	Gmail users are displayed.	Pass
8	User data should be displayed in alphabetical order.	SELECT * FROM Users ORDER BY Username ASC;	Users are sorted by username.	Pass
9	Required fields should not be NULL.	SELECT * FROM Users WHERE Email IS NULL;	No records with NULL email are found.	Pass
10	All user records should be retrievable.	SELECT * FROM Users;	All user records are displayed.	Pass

3. End-to-End Test Scenario: Write one End-to-End test scenario covering: User Registration → Database Entry Validation → Profile Update → Database Update Validation.

Step	Action	SQL Query	Check in DB	Expected Result
1	Laxmi registers as a new user	INSERT INTO Users (Username, Email, Password, Status) VALUES ('Laxmi','laxmi@gmail.com','Laxmi@123','Active');	SELECT * FROM Users WHERE Email='laxmi@gmail.com';	New user record is created with correct values.
2	Check if Laxmi is saved correctly in the database	No separate query – verify result of Step 1	SELECT Username, Status FROM Users WHERE Email='laxmi@gmail.com';	Username = Laxmi, Status = Active.
3	Laxmi updates her username	UPDATE Users SET Username='Laxmi_Rajput' WHERE Email='laxmi@gmail.com';	SELECT Username FROM Users WHERE Email='laxmi@gmail.com';	Username is updated to Laxmi_Rajput.
4	Verify that the update is saved in the database	No separate query – verify result of Step 3	SELECT * FROM Users WHERE Email='laxmi@gmail.com';	Updated username is displayed in the database.
5	Change Laxmi's status to Inactive	UPDATE Users SET Status='Inactive' WHERE Email='laxmi@gmail.com';	SELECT Status FROM Users WHERE Email='laxmi@gmail.com';	Status is updated to Inactive.
6	Final verification of all user details	SELECT * FROM Users WHERE Email='laxmi@gmail.com';	Verify all columns(Username, Email, Password, Status)	All user information is correct and consistent in the database.

4. Test Summary Report: Prepare a Test Summary Report including executed queries, defects identified, and final recommendation.

Overall Testing Summary:

Item	Details
Total Queries Executed	10
Passed	9
Failed	1
Database	phpMyAdmin
Table Tested	user (assessment database)
Tester	Laxmi Rajput
Date	May 2026

Queries Run and Results:

No	Type	Query	Result	Status
1	CREATE	CREATE TABLE user (...)	Table created successfully	PASS
2	INSERT	INSERT INTO user ... x5 rows	5 users added	PASS
3	SELECT	WHERE Status = 'Active'	2 rows returned	PASS
4	SELECT	WHERE Status = 'Inactive'	2 rows returned	PASS
5	SELECT	WHERE Email LIKE '%@gmail.com'	5 rows returned	PASS
6	SELECT	ORDER BY UserName ASC	Sorted correctly	PASS
7	SELECT	WHERE Status='Active' AND Email LIKE '%@gmail.com'	2 rows returned	PASS
8	SELECT	WHERE LastLogin IS NOT NULL	5 rows returned	PASS
9	UPDATE	SET Status='Inactive' WHERE Status='Pending'	1 row updated	PASS
10	DELETE	SELECT * FROM Users WHERE Email IS NOT NULL;	5 rows returned	PASS

Final Recommendation

Overall, the database testing for the **User Management System** was successful. Most queries worked correctly on the first attempt. The **Users** table was created successfully with all required fields and constraints.

- The table structure is correct and all constraints are working properly.
- All **SELECT** queries returned the expected results.
- **UPDATE** and **DELETE** queries worked after adjusting the date conditions.
- It is better to set default values using SQL instead of relying on the GUI.
- Test data dates should be planned carefully so they match the query conditions.
- The database is ready and can be connected to the application after fixing the minor issues mentioned above.